Enunciado Proyecto

Battle Arena Challenge

Plataforma Multijugador Inteligente con Integración Hardware-Software

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE-1105 Principios de Modelado en Ingeniería
II Semestre 2025

1. Contexto del Proyecto

En el mundo actual, los videojuegos competitivos multijugador y la integración de hardware físico con software representan una de las fronteras más interesantes de la ingeniería de software. Las empresas buscan experiencias inmersivas donde los jugadores no solo utilicen teclado y ratón o pantallas táctiles, sino que interactúen mediante dispositivos físicos personalizados (botones, pads o controladores dedicados) que se comunican con el software a través de APIs.

Una startup tecnológica dedicada al desarrollo de experiencias interactivas busca lanzar una nueva plataforma de entretenimiento enfocada en competencias multijugador en tiempo real. La organización desea crear un juego innovador que combine dispositivos físicos con aplicaciones de software modernas, permitiendo que varios usuarios compitan entre sí utilizando controles físicos conectados a través de dispositivos electrónicos y servicios en la nube.

La empresa desea que la solución sea accesible desde computadoras portátiles y dispositivos móviles, permitiendo una experiencia de juego atractiva, competitiva y altamente visual. Además, la plataforma deberá incorporar servicios de autenticación mediante cuentas propias y redes sociales, almacenamiento de estadísticas de juego, clasificación de jugadores, ceremonias de premiación y mecanismos de comunicación con hardware externo.

La dirección de la empresa desea aplicar prácticas modernas de ingeniería de software durante todo el ciclo de vida del proyecto, garantizando calidad, escalabilidad, mantenibilidad y despliegue automatizado.
Los estudiantes actuarán como un equipo profesional de desarrollo encargado de analizar, diseñar y desarrollar una solución funcional que responda a las necesidades planteadas.

Este proyecto permitirá a los estudiantes aplicar de manera integral los conocimientos adquiridos en el curso de Principios de Modelado en Ingeniería, cubriendo desde la elicitación de requerimientos hasta la verificación del software, pasando por arquitectura, patrones de diseño, DevOps, Cloud, contenedores, control de versiones y administración de proyectos.

2. Conversación Inicial con el Stakeholder

La siguiente conversación representa el primer acercamiento con el cliente y deberá servir como insumo principal para las actividades de levantamiento y análisis de requerimientos del Sprint 1.

Stakeholder Principal (Director de Producto):

Nuestra empresa desea lanzar un juego competitivo multijugador en línea. Necesitamos un juego que sea realmente diferente. Queremos algo competitivo, multijugador, que se sienta moderno y que utilice hardware físico. Imaginen un juego en el que cada jugador tenga un pequeño panel de botones físicos que se conectan al software. Esos botones deben mover elementos en pantalla en tiempo real, el juego debe funcionar tanto en computadora como en dispositivos móviles.

Queremos que los jugadores puedan registrarse utilizando una cuenta propia o mediante redes sociales.

Nos interesa que exista algún dispositivo físico externo que permita controlar elementos dentro del juego mediante botones reales.

Los jugadores deberán competir para alcanzar una meta específica dentro de cada partida.

Debe existir un mecanismo que permita declarar ganadores y mostrar algún tipo de ceremonia de victoria al finalizar cada partida.

También queremos almacenar estadísticas históricas y mostrar rankings de los mejores jugadores.

El sistema debe ser moderno, visualmente atractivo y con muchos elementos gráficos.

Idealmente queremos que la aplicación se despliegue utilizando servicios cloud y contenedores para facilitar su distribución.

El sistema debe soportar múltiples jugadores simultáneos.

Además, nos gustaría que todo el proyecto siga prácticas profesionales de desarrollo de software, integración continua y pruebas automatizadas.

Analista de Requerimientos:
¿Cómo imagina la mecánica principal del juego?

Stakeholder:
Debe ser un juego de arena competitiva. Los jugadores se enfrentan en tiempo real. Tiene que haber reglas claras, un sistema de puntuación, determinación de un ganador y una ceremonia de victoria al final. Además, necesitamos que se guarden los mejores récords de partidas anteriores. Que la gente pueda ver quiénes son los mejores jugadores históricos.

Esperamos que el equipo proponga una mecánica innovadora, divertida y desafiante, pero que involucre habilidad, rapidez y estrategia.

Analista:
¿Qué tipo de información desea almacenar de cada jugador?

Stakeholder:
El registro de usuarios es Obligatorio. Cada jugador debe poder crear una cuenta con usuario y contraseña. Idealmente también poder iniciar sesión con redes sociales. Queremos almacenar datos de los jugadores y sus estadísticas.

Se debe tener un nombre de usuario único en el sistema, y este debe estar relacionado con un correo electrónico. Este correo electrónico debe existir en la realidad, por tanto, debe existir un mecanismo para verificar que realmente existe ese correo electrónico. No se permitirán correos no existentes, y solamente una vez se puede registrar dicho correo en el sistema. El mismo correo no debe existir en el registro de más de un jugador.

Entre los mínimos datos requeridos se debe tener: Nombre de usuario, perfil, estadísticas, historial de partidas, logros alcanzados, puntuación acumulada y posición en rankings.

Analista:
¿Cuál es la importancia de los dispositivos físicos?

Stakeholder:
Es uno de los elementos diferenciadores del producto. Queremos que algunas acciones importantes del juego requieran interacción mediante botones físicos conectados mediante APIs o protocolos apropiados.

Los botones deben permitir mover al personaje o a los elementos del juego en pantalla. No queremos solo teclado: queremos que se sienta físico y retador.

Analista:
¿Existe alguna restricción tecnológica?

Stakeholder:
No. El equipo tiene libertad para proponer la arquitectura siempre que pueda justificar técnicamente sus decisiones.

Analista:
¿Qué tan gráfico debe ser?»

Stakeholder:
Muy gráfico. Interfaz atractiva, animaciones, efectos visuales, pantalla de victoria impactante. Que se vea profesional y entretenido. Y por supuesto, que sea lo suficientemente desafiante para que valga la pena competir.

Stakeholder:
Resumiendo: juego multijugador competitivo, control por botones de hardware vía API, versiones para escritorio y móvil, registro de usuarios (local + redes sociales), almacenamiento de récords, reglas claras, ganador y ceremonia de victoria. Eso es lo que necesitamos.

3. Conversación Siguiente con el Stakeholder

Unas semanas después, se realiza nuevamente una elicitación de requisitos, un poco más pausada, y con la definición del producto mucho más "aterrizada" por parte del usuario.

Stakeholder Principal (Director de Producto):
Nuestra empresa desea lanzar un juego competitivo multijugador en línea. Queremos una experiencia que sea entretenida, fácil de comprender para nuevos jugadores, pero suficientemente desafiante para promover la competencia.
La idea general es un juego que llamaremos **Control del Reactor**.
Los jugadores participarán simultáneamente dentro de una arena digital. En el centro de la arena existirá un reactor de energía que genera recursos valiosos durante toda la partida.
El objetivo principal será obtener la mayor cantidad posible de energía proveniente de ese reactor.
Queremos que los jugadores deban controlar personajes o avatares dentro de una arena gráfica y competir entre ellos para acceder al reactor central.
Además, es indispensable que exista integración con un dispositivo de hardware externo mediante botones físicos conectados al sistema.

Analista:
¿Cómo imagina la mecánica principal del juego?

Stakeholder:
Todos los jugadores participan dentro de una única arena.
La arena representa una instalación tecnológica donde se encuentra un reactor en el centro.
El reactor produce energía periódicamente.
Para obtener esa energía, los jugadores deben desplazarse por la arena y llegar al reactor antes que los demás.
No queremos que simplemente se acerquen y obtengan puntos automáticamente.
Debe existir una acción específica que requiera interacción mediante un botón físico para extraer la energía del reactor.
Mientras un jugador está extrayendo energía se encuentra expuesto a perder oportunidades frente a otros competidores.
Esperamos que exista una combinación de rapidez, estrategia y toma de decisiones.

Analista:
¿Qué obstáculos tendrán los jugadores durante la partida?

Stakeholder:
No queremos un escenario vacío.
La arena debe contener obstáculos que dificulten el acceso al reactor.
Por ejemplo:

- Barreras móviles.
- Puertas automáticas.
- Corredores que se bloquean temporalmente.
- Trampas que reduzcan la velocidad de movimiento.
- Sectores de riesgo.

Además, deseamos que existan guardianes automáticos que protejan el reactor.
Pueden moverse siguiendo rutas predefinidas o patrones sencillos.
Si un guardián alcanza a un jugador, debería existir alguna penalización.
Por ejemplo:

- Pérdida de energía acumulada.
- Reducción temporal de velocidad.
- Regreso al punto de inicio.

Analista:
¿Cómo funcionará la obtención de puntos?

Stakeholder:
La principal fuente de puntos debe ser la energía obtenida desde el reactor central.
Sin embargo, queremos algunas actividades secundarias que permitan generar puntuación adicional.
Por ejemplo:

- Activación de terminales energéticas.
- Recolección de celdas de energía distribuidas por la arena.
- Bonificaciones temporales.
- Cumplimiento de eventos especiales.

Los jugadores deben asumir riesgos para conseguir las mayores recompensas.
Las acciones más sencillas deben otorgar pocos puntos.
Las acciones más difíciles deben otorgar muchos más puntos.

Analista:
¿Cómo desea que se desarrolle la competencia entre los jugadores?

Stakeholder:
Queremos que todos compitan por el mismo objetivo.
El reactor debe ser el punto principal de interés durante toda la partida.
No deseamos que los jugadores se distribuyan por zonas aisladas del mapa.
La competencia debe concentrarse cerca del reactor y sus alrededores.
Debe sentirse como una lucha constante por controlar el acceso a ese recurso.

Analista:
¿Desea múltiples escenarios o mapas?

Stakeholder:
Sí.
Nos gustaría que cada partida pueda desarrollarse en un escenario diferente.
Por ejemplo:

- Laboratorio subterráneo.
- Estación espacial.
- Planta industrial.
- Centro de investigación.
- Complejo militar futurista.

Sin embargo, las reglas deben mantenerse iguales.
Lo que cambia es la distribución de obstáculos, rutas y guardianes.

Analista:
¿Qué importancia tienen los dispositivos físicos?

Stakeholder:
Son un componente fundamental del proyecto.
No queremos que todo pueda realizarse únicamente con teclado, mouse o pantalla táctil.
El sistema debe incluir un panel físico de botones conectado mediante APIs o protocolos apropiados.
Los botones deben permitir controlar el movimiento del personaje.
Además, debe existir al menos una acción importante del juego que dependa de dichos botones.
Por ejemplo:

- Extraer energía del reactor.
- Activar terminales.
- Desbloquear puertas especiales.
- Utilizar bonificaciones.

La interacción física debe sentirse relevante para la experiencia de juego.

Analista:
¿Cuáles son los controles mínimos requeridos?

Stakeholder:
Como mínimo esperamos cinco botones físicos:

- Arriba.
- Abajo.
- Izquierda.
- Derecha.
- Acción.

Los cuatro primeros permiten el desplazamiento.

El botón de acción deberá utilizarse para interactuar con los elementos importantes de la arena.

Analista:
¿Cómo desea que funcionen los eventos especiales?

Stakeholder:
Queremos que el juego sea dinámico.
Cada cierto tiempo pueden ocurrir eventos aleatorios.
Por ejemplo:

- Sobrecarga del reactor.
- Aparición de energía adicional.
- Incremento temporal de puntuación.
- Desactivación de ciertas barreras.
- Aparición de nuevos guardianes.
- Apertura temporal de rutas alternativas.

Estos eventos deben modificar temporalmente el comportamiento de la partida.

Analista:
¿Qué información desea almacenar de cada jugador?

Stakeholder:
El registro de usuarios es obligatorio.
Cada jugador debe poseer un nombre de usuario único.
Además, cada cuenta debe estar asociada a un correo electrónico válido.
No deben existir correos repetidos dentro del sistema.
Queremos almacenar como mínimo:

- Nombre de usuario.
- Correo electrónico.
- Perfil del jugador.
- Historial de partidas.
- Estadísticas acumuladas.
- Total de puntos obtenidos.
- Cantidad de victorias.
- Posición en rankings.
- Logros alcanzados.

También nos interesa conservar récords históricos de todas las partidas realizadas.

Analista:
¿Cómo se determina el ganador?

Stakeholder:
El ganador debe determinarse automáticamente.

La partida puede finalizar cuando:

- Se alcance un tiempo límite.
- O se alcance una puntuación objetivo previamente definida.

Al finalizar, el sistema debe ordenar automáticamente a los participantes según los puntos obtenidos.
Debe existir:

- Primer lugar.
- Segundo lugar.
- Tercer lugar.

Además, debe actualizarse el ranking global del sistema.

Analista:
¿Cómo imagina la ceremonia de victoria?

Stakeholder:
La ceremonia debe ser visualmente atractiva.
Queremos una pantalla especial donde aparezcan:

- El ganador de la partida.
- El podio de los mejores jugadores.
- Resumen de estadísticas.
- Récords alcanzados.

También esperamos:

- Animaciones.
- Efectos visuales.
- Confeti.
- Trofeos.
- Sonidos de celebración.

Debe sentirse como una recompensa importante para el jugador.

Analista:
¿Qué tan gráfica debe ser la aplicación?

Stakeholder:
Muy gráfica.
La arena debe contener elementos visuales modernos.
Queremos:

- Animaciones.
- Efectos de partículas.
- Indicadores de puntuación.
- Temporizadores.
- Tablas de posiciones en tiempo real.
- Interfaz moderna y atractiva.

La experiencia visual es un aspecto clave del producto.

Analista
¿Existe alguna restricción tecnológica?

Stakeholder:
No.
El equipo tiene libertad para proponer la arquitectura tecnológica.
Lo importante es que puedan justificar adecuadamente las decisiones tomadas.
Nos gustaría además que el sistema:

- Utilice bases de datos para persistencia.
- Permita múltiples jugadores simultáneos.
- Utilice servicios cloud para despliegue.
- Emplee contenedores cuando sea posible.
- Incorpore integración continua.
- Considere pruebas automatizadas.
- Siga buenas prácticas profesionales de desarrollo de software.

Stakeholder:
En resumen, deseamos un juego competitivo multijugador en tiempo real llamado **Control del Reactor**, donde los jugadores compiten dentro de una arena para obtener energía de un reactor central utilizando controles físicos conectados mediante hardware externo.
El juego deberá incluir:

- Registro de usuarios.
- Inicio de sesión.
- Integración con hardware físico.
- Obstáculos y guardianes.
- Múltiples escenarios.
- Sistema de puntuación.
- Eventos especiales.
- Determinación automática de ganador.
- Ranking global.
- Historial de partidas.
- Ceremonia de victoria.
- Amplio contenido gráfico.
- Compatibilidad con computadora y dispositivos móviles.
- Persistencia de datos.
- Arquitectura moderna basada en buenas prácticas de ingeniería de software.

CONTROL DEL REACTOR

COMPITE. CONTROLA. CONQUISTA.

Enfréntate a otros jugadores en una arena futurista para controlar el reactor central y obtener la máxima energía. Usa tus habilidades, estrategia y reflejos para ser el mejor.



OBJETIVO PRINCIPAL

Obtener la mayor cantidad de energía extraída del reactor antes de que termine el tiempo o se alcance la puntuación objetivo.

CARACTERÍSTICAS PRINCIPALES

- **MULTIJUGADOR EN LÍNEA** — Compite en tiempo real con otros jugadores.
- **ÚNICO OBJETIVO CENTRAL** — Todos luchan por controlar el reactor.
- **ACCIÓN Y ESTRATEGIA** — Toma riesgos, evita obstáculos y supera a tus rivales.
- **CONTROL CON HARDWARE** — Usa botones físicos conectados al juego.
- **GANADOR Y CEREMONIA** — El sistema determina al ganador y celebra la victoria.
- **RANKING Y ESTADÍSTICAS** — Historial de partidas y ranking global de los mejores jugadores.

JUGABILIDAD

- **LLEGA AL REACTOR** — Compite para ser el primero en llegar al centro.
- **EXTRAE ENERGÍA** — Mantén presionado el botón de acción para extraer energía del reactor.
- **DEFIÉNDETE Y SUPERA OBSTÁCULOS** — Evita trampas, guardianes y barreras que dificultan el acceso.
- **OBTÉN PUNTOS ADICIONALES** — Activa terminales, recoge celdas de energía y aprovecha los eventos especiales.

ELEMENTOS EN LA ARENA



EVENTOS ESPECIALES

Cada cierto tiempo ocurren eventos que cambian el ritmo de la partida.

- **SOBRECARGA** — El reactor genera el doble de energía.
- **INVASIÓN** — Aparecen más guardianes en la arena.
- **APAGÓN** — Parte del escenario queda a oscuras.
- **RUTA ALTERNATIVA** — Se abren caminos secretos temporalmente.
- **LLUVIA DE ENERGÍA** — Aparecen más celdas de energía.

CONTROLES CON HARDWARE



Los botones físicos se conectan al sistema mediante API. El botón de acción permite extraer energía, activar terminales, abrir puertas y usar bonificaciones.



Ejemplo de conexión ESP32 / Arduino

Servidor del Juego

PUNTUACIÓN

Energía del reactor	20 - 50 pts
Celdas de energía	10 pts
Terminales activadas	25 pts
Bonificaciones	15 - 50 pts
Eventos especiales	20 - 100 pts

La partida termina cuando:
- Se agota el tiempo límite (ej. 5 minutos)
- Algún jugador alcanza la puntuación objetivo

El sistema ordena automáticamente a los jugadores y determina al ganador.

ESCENARIOS DINÁMICOS

Cada partida se desarrolla en un escenario diferente.





Las reglas son las mismas, lo que cambia es la distribución de obstáculos, rutas, trampas y guardianes.

PERFIL Y PROGRESIÓN

- Registro con usuario único y correo electrónico verificado.
- Estadísticas personales.
- Historial de partidas.
- Logros y medallas.
- Posición en ranking global.



ALEX_07

Nivel 15 — 3500 / 6000 XP

ESTADÍSTICAS

Partidas jugadas	125
Victorias	78
Energía total obtenida	58,320
Mejor puntuación	1,250
Posición en ranking	#23

CEREMONIA DE VICTORIA



TROFEOS — EFECTOS VISUALES — ESTADÍSTICAS DE LA PARTIDA — RÉCORDS PERSONALES

ARQUITECTURA GENERAL



TECNOLOGÍAS SUGERIDAS

- **FRONTEND** — React / Flutter
- **BACKEND** — Node.js / ..NET
- **TIEMPO REAL** — WebSocket / Socket.IO
- **BASE DE DATOS** — PostgreSQL
- **CLOUD & CONTENEDORES** — Docker / Kubernetes / AWS / Azure
- **INTEGRACIÓN CONTINUA** — GitHub Actions / GitLab CI

4. Componentes Mínimos Obligatorios

Gestión de Usuarios

El sistema deberá permitir:

- Registro de usuarios.
- Inicio de sesión.
- Validación de correo electrónico
- Recuperación de contraseña.
- Administración de perfiles.

Autenticación Social

Integración con al menos una red social:

- Google
- Facebook
- Microsoft
- GitHub

Juego Multijugador

Debe incluir:

- Mínimo 2 jugadores simultáneos.
- Competencia directa.
- Reglas claramente definidas.
- Sistema de puntuación.
- Temporizador de partida.
- Detección automática de ganador.

Ranking Histórico

Debe almacenarse:

- Top 10 jugadores.
- Récords históricos.
- Estadísticas individuales.
- Historial de partidas.

Ceremonia de Ganador

Al finalizar cada partida:

- Presentación visual del ganador.
- Animaciones.
- Sonidos o efectos visuales.
- Actualización automática del ranking.

Integración Hardware

El proyecto deberá utilizar hardware físico.
Ejemplos:

- ESP32
- Arduino
- Raspberry Pi Pico
- Raspberry Pi

Pudiendo incorporar:
- Botones físicos
- LEDs
- Sensores
- Joysticks

La comunicación debe realizarse mediante APIs o protocolos apropiados:
- REST
- WebSocket
- MQTT
- Bluetooth
- Serial

Base de Datos
Debe almacenar:
- Usuarios
- Credenciales
- Partidas
- Puntajes
- Eventos
- Rankings

Aplicación Móvil
Debe permitir como mínimo:
- Inicio de sesión
- Consulta de estadísticas
- Participación en partidas
- Visualización de rankings

Patrones de diseño que se pueden aplicar
Factory
Crear Guardianes, eventos y bonificaciones.
Strategy
Permitir distintos comportamientos de guardianes.
Observer
Actualizar puntajes y rankings en tiempo real.
State

Estados del jugador:

- Normal-Activo
- Escudado
- Aturdido
- Eliminado temporalmente

Repository

Persistencia de usuarios y partidas.

Mecánica general sugerida

Los jugadores se enfrentan en una arena. Cada uno controla su entidad mediante botones físicos (mover, acción especial, defensa, etc.). El objetivo es acumular la mayor cantidad de puntos según las reglas establecidas. Al finalizar el tiempo o al cumplirse la condición de victoria, se declara un ganador y se muestra una ceremonia de victoria. Los mejores puntajes se almacenan y se muestran en un ranking histórico.

5. Objetivos del Proyecto

Objetivo general

Desarrollar un producto de software completo (juego multijugador competitivo con integración de hardware) aplicando de forma integral los principios de modelado, arquitectura, desarrollo, operaciones y verificación de software.

Objetivos específicos

1. Realizar la elicitación, análisis y especificación de requerimientos a partir de la conversación con el Stakeholder y técnicas complementarias.
2. Diseñar y documentar la arquitectura del producto considerando escritorio, móvil y comunicación con hardware.
3. Aplicar principios de desarrollo de software y patrones de diseño (creacionales, estructurales y de comportamiento).
4. Implementar control de versiones, prácticas de administración de proyectos y flujos DevOps.
5. Utilizar virtualización, contenedores y servicios de cloud computing cuando sea pertinente.
6. Realizar verificación y pruebas de software de manera sistemática.
7. Entregar un producto funcional que cumpla con las características descritas.

Temas del Curso que Deben Aplicarse

Durante el desarrollo del proyecto deberán aplicarse los siguientes conocimientos:

Elicitación de Requerimientos

- Entrevistas

- Historias de usuario
- Casos de uso
- Priorización

Levantamiento de Arquitectura
- Arquitectura lógica
- Arquitectura física
- Diagramas UML
- Diagramas de despliegue
- Diagramas de componentes

DevOps
- CI/CD
- Pipelines automatizados
- Build automático

Cloud Computing
- Despliegue en nube pública o privada

Virtualización y Contenedores
- Docker obligatorio
- Docker Compose o Kubernetes deseable

Administración de Proyectos
- Backlog
- Sprint Planning
- Sprint Review
- Gestión de riesgos

Control de Versiones
- Git obligatorio
- Estrategia de ramas documentada

Principios de Desarrollo de Software
- SOLID
- DRY
- Clean Code

Patrones de Diseño

Al menos:
- 1 patrón creacional

- 1 patrón estructural
- 1 patrón de comportamiento

Verificación y Pruebas
- Unitarias
- Integración
- Aceptación
- Automatizadas

6. Descripción de la Metodología de Desarrollo del Proyecto

El propósito principal de este proyecto no es únicamente desarrollar un videojuego funcional denominado **Control del Reactor**, sino permitir que los estudiantes experimenten de forma práctica el ciclo completo de desarrollo de software utilizando:
- Ingeniería de Requisitos.
- Metodologías Ágiles.
- Scrum.
- Buenas prácticas de diseño de software.
- Desarrollo orientado a pruebas.
- DevOps.
- Gestión de configuración.
- Integración continua.
- Entrega continua.
- Documentación profesional.

El proyecto busca simular un entorno de desarrollo real donde los estudiantes asumen diferentes roles dentro de un equipo de trabajo y deben tomar decisiones técnicas, organizacionales y de planificación a lo largo de varios Sprints.

El proyecto será desarrollado utilizando el marco de trabajo **Scrum**.
Durante el desarrollo se espera que los estudiantes apliquen adecuadamente:
- Product Backlog.
- Sprint Backlog.
- User Stories.
- Priorización de requisitos.
- Planificación iterativa.
- Entregas incrementales.
- Refinamiento continuo.
- Gestión de tareas mediante Azure DevOps.

El proyecto se desarrolla en tres Sprints. Cada Sprint tiene objetivos claros y entregables obligatorios.

Sprint 1: Análisis, Comprensión y Planificación

Presentación Enunciado	Fecha Entrega
3 septiembre, 2026 (S5)	24 septiembre, 2026 (S8)

Objetivo

El primer Sprint es completamente analítico.
Durante este Sprint no se espera una implementación significativa del producto.
El enfoque principal será comprender profundamente el problema planteado, identificar necesidades del cliente y construir una planificación sólida para el resto del proyecto.

Actividades Esperadas

1. **Elicitación y análisis de requisitos**

A partir de la entrevista realizada al Stakeholder Principal, el equipo deberá:

- Identificar requisitos funcionales.
- Identificar requisitos no funcionales.
- Identificar reglas de negocio.
- Identificar restricciones tecnológicas.
- Detectar posibles ambigüedades.
- Elaborar preguntas de refinamiento.

2. **Definición de Historias de Usuario**

Todos los requisitos encontrados en las entrevistas deberán transformarse en **User Stories**.
Ejemplo:

US-001
Como jugador
Quiero registrarme utilizando un correo electrónico válido
Para poder participar en partidas del juego.

3. **Priorización del Product Backlog**

Cada Historia de Usuario deberá ser priorizada utilizando técnicas apropiadas.
Por ejemplo:

- Alta prioridad.
- Media prioridad.
- Baja prioridad.

o bien:

- Must Have.
- Should Have.
- Could Have.
- Won't Have.

4. **Definición de Criterios de Aceptación**

Cada Historia deberá poseer criterios de aceptación claros y verificables.
Ejemplo:

Criterios de aceptación

- El correo debe ser único.
- El correo debe ser validado.
- El usuario debe poder iniciar sesión posteriormente.

5. **Casos de Uso**

- Presentar cómo mínimo 10 Casos de uso debidamente construidos.
    - Dependencias
    - Precondición
    - Descripción
    - Secuencia normal
    - PostCondición
    - Excepciones
    - Comentarios
    - Mostrar diagrama UML de casos de uso

6. **Definición de Definition of Done (DoD)**

Cada tarea deberá poseer una **Definición de Done** claramente documentada.
Ejemplo:
Una tarea se considera completada cuando:

- El código ha sido implementado.
- Existen pruebas unitarias.
- Las pruebas ejecutan correctamente.
- El código ha sido revisado.
- La documentación ha sido actualizada.
- El cambio ha sido integrado al repositorio.

7. **Planificación Inicial de los Sprints**

El equipo deberá definir una hoja de ruta preliminar de los primeros cinco Sprints.
Se espera que identifiquen:

- MVP 1
- MVP 2
- MVP 3
- MVP 4
- Producto Final

Cada Sprint debe producir un incremento funcional del producto. Se debe documentar esto, y será parte de los entregables del Sprint.

8. **Análisis de la Arquitectura propuesta**

Teniendo en cuenta los requisitos planteados y las buenas prácticas de desarrollo, plantear una "arquitectura ideal" que permita desarrollar el producto de la mejor manera posible. No necesariamente se llevará a cabo dicha Arquitectura, pero la idea es tener una concepción ideal para que todo resulte de la mejor manera.
Para efectos de esta tarea, se debe levantar un "diagrama de arquitectura" debidamente estructurado.

9. **Bitácora**

- Deben levantar una bitácora del trabajo realizado por cada uno de los integrantes del equipo.
- Al final de la bitácora debe existir una tabla en la cual se encuentre el nombre de cada integrante del equipo y el porcentaje de aportes realizado en general. La sumatoria deberá ser de 100%.

10. **Video Explicativo de la entrega**

Se debe entregar un video en el cual deben participar los integrantes del equipo que trabajaron en este primer Sprint.
El video debe ser máximo de 15 minutos, y será utilizado como forma del Sprint Review para formalizar la entrega del Producto Mínimo Viable al Product Owner (en este caso al profesor).
En el video deben presentar, en este caso, todos y cada uno de los entregables generados en este primer Sprint. El que aparezca en el video, no significa que el profesor no lo revisará, por tanto, para efectos de la evaluación se revisará cada una de las entregas en su debida documentación dentro del Wiki de Azure DevOps.

Entregables Esperados Sprint 1

- Product Backlog.
- Historias de Usuario.
- Clasificación de los requisitos (funcionales, no funcionales, etc.)
- Priorización.
- Criterios de aceptación.
- Casos de Uso
- Definition of Done.
- Arquitectura propuesta.
- Modelo de dominio inicial.
- Planificación de Sprints.
- Riesgos identificados.
- Wiki actualizada.
- Bitácora.

Para efectos de la entrega se debe recopilar los entregables esperados en el **documento ERS/SRS.** En el anexo a este enunciado se presenta un ejemplo de este documento, pero es responsabilidad del equipo investigar y entregar un documento apropiado.

Sprint 2: Primera Implementación

Presentación Enunciado	Fecha Entrega
24 septiembre, 2026 (S8)	22 octubre, 2026 (S12)

Objetivo
Desarrollar el primer conjunto de funcionalidades priorizadas por el Product Owner.

Se les entregará los requisitos debidamente priorizados con sus correspondientes "criterios de aceptación" que serán usados para la validación del producto mínimo viable solicitado.

Expectativas
Las funcionalidades implementadas deberán seguir buenas prácticas profesionales.
No se evaluará únicamente si funcionan.
También se evaluará la calidad de la solución.

Aspectos Obligatorios
Desarrollo

- Programación orientada a objetos.
- Patrones de diseño apropiados.
- Principios SOLID.
- Separación por capas.
- Manejo adecuado de errores.
- Arquitectura seleccionada apropiada

Control de versiones
Se deberá utilizar:

- Git
- Repositorios remotos
- Estrategia de ramas adecuadamente definida

Persistencia
Se deberá utilizar una base de datos (ej. Postgres)
No se aceptarán soluciones que almacenen toda la información únicamente en memoria.

Contenedores
Se deberá utilizar Docker para la ejecución y despliegue de componentes relevantes.

DevOps
Se deberá configurar como mínimo:

- Build automático.

- Integración continua.
- Ejecución automática de pruebas.

Actividades obligatorias del Sprint 2

1. Diseñar la arquitectura del producto (vista lógica, de despliegue, de componentes, etc.).
2. Seleccionar y justificar patrones de arquitectura y patrones de diseño aplicables.
3. Definir la estrategia de control de versiones y estructura del repositorio.
4. Planificar el proyecto (cronograma, roles, riesgos, herramientas de administración).
5. Diseñar el modelo de datos (jugadores, partidas, récords).
6. Definir la comunicación con hardware mediante APIs.
7. Implementar el esqueleto del sistema (autenticación básica, estructura de proyecto, conexión inicial con hardware o su simulación).
8. Configurar entorno de desarrollo y prácticas iniciales de DevOps (CI básico).

Entregables Esperados Sprint 2

- MVP funcional.
- Repositorio Git.
- Contenedores Docker.
- Base de datos integrada.
- Pipeline DevOps inicial.
- Wiki actualizada.

Sprint 3: Evolución del Producto

Presentación Enunciado	Fecha Entrega
22 octubre, 2026 (S12)	23 noviembre, 2026 (S18)

Objetivo

Incorporar nuevos requisitos entregados por el profesor.
Estos requisitos representarán cambios y nuevas necesidades del cliente.
Se busca que los estudiantes experimenten situaciones reales de evolución de software.
Entregar una versión completamente funcional del juego.

Consideraciones

Los equipos podrán:

- Implementar nuevos requisitos.
- Completar requisitos pendientes.
- Corregir defectos detectados en Sprints anteriores.
- Refactorizar componentes cuando sea necesario.

Aspectos Evaluados

- Capacidad de adaptación.
- Calidad de integración.
- Gestión del Product Backlog.
- Trazabilidad de requisitos.

Actividades Esperadas

Integración

- Integración completa de componentes.
- Integración con base de datos.
- Integración con hardware físico.
- Integración entre clientes y servidor.

Pruebas

- Pruebas funcionales.
- Pruebas de integración.
- Pruebas de aceptación.
- Pruebas de regresión.

Corrección de defectos

- Atención de incidencias.
- Estabilización del producto.

Resultado Esperado

Se espera poder realizar una partida completa donde participen al menos dos jugadores reales.
La demostración deberá incluir:

- Registro.
- Inicio de sesión.
- Conexión al juego.
- Interacción mediante hardware.
- Sistema de puntuación.
- Determinación de ganador.
- Ranking.
- Pantalla de victoria.

DevOps y Calidad

Durante todo el proyecto se espera la aplicación de buenas prácticas DevOps.

Evidencia Esperada

CI/CD

- Compilación automática.
- Ejecución automática de pruebas.
- Validación automática de calidad.

Control de Versiones

- Commits frecuentes.
- Mensajes descriptivos.
- Historial verificable.

Automatización

- Scripts reproducibles.
- Configuración versionada.

Documento de Pruebas

La evaluación considerará fuertemente el nivel de calidad del proceso de pruebas.
Debe existir un documento formal que incluya:

Casos de prueba

Por cada Historia de Usuario atendida durante los Sprint:

- Identificador.
- Objetivo.
- Precondiciones.
- Datos de entrada.
- Resultado esperado.
- Resultado obtenido.

Escenarios de prueba

Ejemplos:

Escenario 1

Registro exitoso de usuario.

Escenario 2

Intento de registro con correo duplicado.

Escenario 3

Inicio de sesión correcto.

Escenario 4

Extracción de energía utilizando hardware.

Escenario 5

Finalización de partida y actualización del ranking.

Documentación Obligatoria

Toda la documentación del proyecto deberá mantenerse utilizando la **Wiki de Azure DevOps**.
En cada Sprint deberá actualizarse como mínimo:

- Avances.
- Decisiones técnicas.
- Arquitectura.
- Historias de usuario.
- Evidencias de pruebas.
- Retrospectivas.
- Riesgos.
- Lecciones aprendidas.

Uso de Azure DevOps Boards

El uso adecuado de los Boards es un requisito obligatorio y de alta ponderación dentro de la evaluación.
Cada equipo deberá registrar:

- Historias de Usuario.
- Features.
- Tasks.
- Bugs.
- Impedimentos.

Asimismo, deberán:

- Asignar responsables.
- Estimar esfuerzo.
- Actualizar estados.
- Mantener trazabilidad entre requisitos y tareas.

Criterio de Evaluación Relevante

Se calificará no solamente el producto final, sino también:

1. La calidad del análisis de requisitos.
2. La correcta aplicación de Scrum.
3. La gestión del Product Backlog.
4. La documentación mantenida en Azure DevOps.
5. El uso de Boards.
6. La evidencia de trabajo colaborativo.
7. La aplicación de DevOps.
8. La calidad de las pruebas.
9. El uso de buenas prácticas de arquitectura y diseño.
10. La calidad y completitud del juego desarrollado.

Evaluación

La evaluación del proyecto se realizará de forma continua a lo largo de los tres Sprints. Cada Sprint tendrá una entrega y una revisión. La ponderación específica será definida por el profesor, pero se evaluarán de manera prioritaria:

- Calidad y completitud de la elicitación y especificación de requisitos (Sprint 1).
- Calidad de la arquitectura y del diseño, uso justificado de patrones, y producto mínimo viable presentado (Sprint 2).
- Funcionalidad del producto, integración hardware, contenido gráfico y experiencia de juego (Sprint 3).
- Aplicación real de DevOps, control de versiones, contenedores/cloud y verificación.
- Calidad de la documentación y de las demostraciones.
- Trabajo en equipo y administración del proyecto.

Rúbrica de Evaluación del Proyecto Integrador
Valor Total del Proyecto: 50% de la Nota Final del Curso

El proyecto se desarrollará mediante una metodología incremental basada en tres Sprints. Cada Sprint será evaluado de forma independiente y contribuirá a la nota final del proyecto de acuerdo con la siguiente distribución:

Sprint	Valor sobre el curso	Peso dentro del proyecto
Sprint 1: Análisis, Comprensión y Planificación	10 %	20 %
Sprint 2: Primera Implementación	15 %	30 %
Sprint 3: Evolución del Producto	25 %	50 %
TOTAL PROYECTO	**50 %**	**100 %**

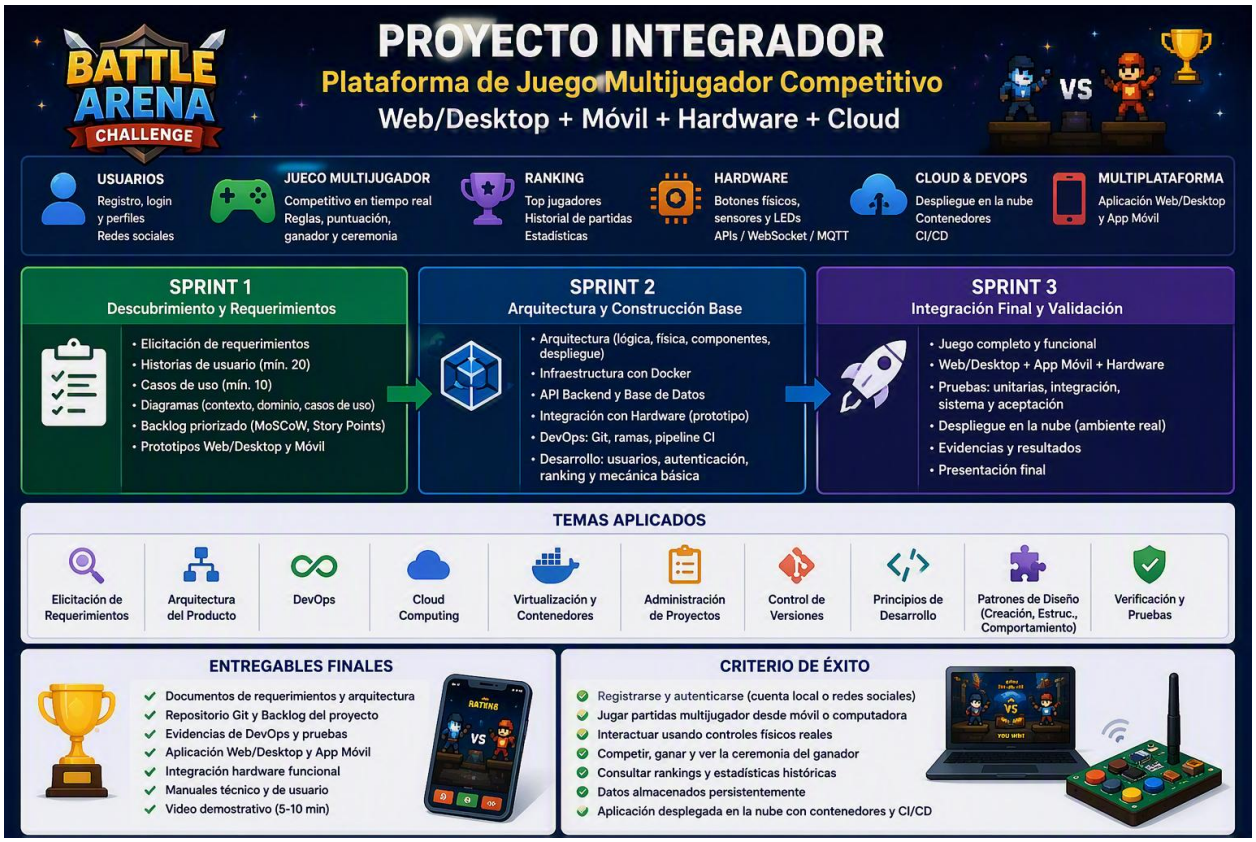
Cada Sprint se evalúa sobre 100 puntos. La nota obtenida se convierte proporcionalmente a su porcentaje correspondiente de la nota final. En los siguientes enunciados de Sprint se detallarán los requerimientos solicitados, así como la evaluación para ese especifico sprint.

Rúbrica Sprint 1 – Análisis, Comprensión y Planificación (10 % de la nota final)

Criterio	Descripción	Excelente	Bueno	Aceptable	Insuficiente	Pts
Product Backlog (clasificación y priorización)	Claridad, completitud, clasificación (funcionales/no funcionales) y formulaciones correctas.	Requisitos claros, completos y bien clasificados (18-20)	Buen nivel de claridad y cobertura (15-18)	Requisitos incompletos o ambiguos (12-15)	Especificación pobre o incorrecta (0-10)	**20**
User Histories & Criterios de aceptación	Cumplimiento con las buenas prácticas, claros, concisos y con criterios de aceptación correctos	Excelente calidad y criterios medibles (14-15)	Buenos user stories y criterios de aceptación (11-13)	US básicos o incompletos (8-10)	Deficiente o ausente (0-7)	**15**
Casos de uso	Cumple con los elementos esenciales de Casos de uso.	Excelente calidad y criterios medibles (14-15)	Buenos casos con criterios (11-13)	Casos de uso básicos o incompletos (8-10)	Deficiente o ausente (0-7)	**15**
Definition of Done	Correcta y adecuada definición de done de las tareas.	Excelente y completo (5-4)	Incompleta definición (4-3)	Incompleta e incorrecta (2-1)	Deficiente o ausente (0)	**5**
Arquitectura Propuesta del Proyecto completo	Cumple con los aspectos requeridos y componentes de una arquitectura funcional, escalable y correcta.	Excelente y completo (9-10)	Incompleta definición (7-8)	Incompleta e incorrecta (5-6)	Deficiente o ausente (0-4)	**10**
Riesgos Identificables	Se identifican riesgos tanto de negocio, técnicos, internos, y externos que pueden afectar al proyecto.	Excelente identificación de riesgos (5-4)	Incompleta identificación de riesgos (4-3)	Incompleta e incorrecta (2-1)	Ausente o incorrecta (0)	**5**
Calidad del documento ERS/SRS	Organización, redacción, profesionalismo y presentación del documento de requisitos.	Excelente redacción, presentación y completitud (9-10)	Problemas de redacción e incompleto (7-8)	Incompleto e incorrecto (5-6)	Documento deficiente (0-4)	**10**
Wiki Actualizada	Organización, redacción, profesionalismo y presentación de documentos.	Documento profesional y bien estructurado (5-4)	Buen documento con detalles menores (4-3)	Documento aceptable pero incompleto (2-1)	Documento deficiente (0)	**5**
Bitácora	Se presenta una adecuada y completa bitácora.	Documento completo (5-4)	Documento incompleto (4-3)	Documento incorrecto (2-1)	Documento definiciente (0)	**5**
Video Explicativo de la Entrega	Se presenta un adecuado video explicativo de entregables	Video completo y abarca todo lo solicitado. (9-10)	Video Incompleto. (7-8)	Video incompleto e incorrecto. (5-6)	Video ausente o incorrecto. (0-4)	**10**

Recomendaciones finales

- Utilicen herramientas de gestión de proyectos (tablero Kanban/Scrum) en Azure.
- Mantengan el repositorio ordenado y con commits significativos (use Git).
- Documenten las decisiones de arquitectura y de diseño (ADR – Architecture Decision Records).
- Prioricen un producto mínimo viable (MVP) que sea demostrable y luego incrementen funcionalidades.
- Prueben temprano la comunicación con el hardware o su simulación.
- Mantengan comunicación constante con el profesor en el rol de Stakeholder.



Anexo

Para este proyecto, el documento **ERS (Especificación de Requisitos de Software)** o **SRS (Software Requirements Specification)** debería ser uno de los entregables más importantes del **Sprint 1**, ya que representa la consolidación formal de todo el trabajo de elicitación, análisis y planificación realizado por el equipo.

Objetivo del ERS/SRS
El objetivo del documento es describir de manera clara, completa, verificable y sin ambigüedades qué debe hacer el sistema antes de iniciar el desarrollo.

Debe convertirse en el principal acuerdo entre:
- Stakeholder
- Product Owner
- Equipo de desarrollo
- Profesor

Estructura Recomendada del ERS/SRS
1. Introducción
1.1 Propósito
Descripción del objetivo del documento.

Ejemplo:
Este documento describe los requisitos funcionales y no funcionales del sistema "Control del Reactor", un videojuego multijugador competitivo que integra hardware físico, aplicaciones cliente y servicios backend.

1.2 Alcance
Debe describir:
- Qué es el sistema.
- Qué problema resuelve.
- A quién está dirigido.

1.3 Definiciones y Acrónimos
Ejemplos:

Término	Descripción
API	Interfaz de programación
MVP	Producto Mínimo Viable
CI/CD	Integración y Entrega Continua
PO	Product Owner

DoD	Definition of Done

2. Descripción General

2.1 Perspectiva del Producto

Diagrama de contexto.

2.2 Características Generales

Resumen de funcionalidades

2.3 Tipos de Usuarios

Por ejemplo:

Jugador

- Participa en partidas.
- Consulta rankings.

Administrador

- Gestiona la plataforma.
- Consultas estadísticas.

2.4 Restricciones

Ejemplos:

- Uso obligatorio de Azure DevOps.
- Uso obligatorio de Git.
- Uso obligatorio de Docker.
- Uso de Base de Datos.
- Compatibilidad desktop.

3. Requisitos Funcionales

Esta sección debe ser extremadamente detallada.
Cada requisito debe tener identificador.
Ejemplo:

RF-001 Registro de Usuario

Descripción:
El sistema permitirá registrar jugadores.
Entradas:

- Usuario.
- Correo.
- Contraseña.

Resultado:

- Cuenta creada.

RF-002 Inicio de Sesión

Descripción:
El sistema deberá autenticar jugadores registrados.

RF-003 Crear Partida

RF-004 Unirse a Partida

Y así sucesivamente.

4. Historias de Usuario
Cada requisito debe vincularse con una historia.
Ejemplo:
US-01
Como jugador
Quiero registrarme en el sistema
Para poder participar en partidas.

Criterios de aceptación
- Correo obligatorio.
- Correo único.
- Confirmación exitosa.

Prioridad
Alta

Sprint
Sprint 2

5. Requisitos No Funcionales
Muy importantes para este proyecto.

Rendimiento
RNF-01
El sistema deberá soportar al menos dos jugadores simultáneos.

RNF-02
La actualización de movimientos deberá reflejarse en menos de 500 ms.

Seguridad
RNF-03
Las contraseñas deberán almacenarse cifradas.

RNF-04
No se permitirán correos duplicados.

Disponibilidad
RNF-05
El sistema deberá ejecutarse mediante contenedores Docker.

Escalabilidad
RNF-06
La arquitectura deberá permitir agregar nuevas funcionalidades.

6. Reglas de Negocio
Ejemplo:
RN-01
Cada correo electrónico sólo puede pertenecer a un jugador.

RN-02
Gana el jugador con más energía acumulada.

RN-03
Un guardián puede penalizar jugadores.

RN-04
Un jugador no puede estar en más de una partida simultáneamente.

7. Casos de Uso
Ejemplo:
CU-01 Registrarse
Actores:
- Jugador

Flujo Principal:
1. Ingresa datos.
2. Verifica correo.
3. Crea cuenta.

Flujos alternos:
- Correo repetido.
- Correo inválido.

8. Modelo de Dominio
Diagrama conceptual con entidades:

9. Arquitectura Propuesta
Debe incluir diagramas:

- **Arquitectura lógica**
- **Arquitectura de despliegue**

10. Product Backlog Inicial
Tabla con:

- ID
- Historia
- Prioridad
- Sprint estimado

Ejemplo:

ID	Historia	Prioridad	Sprint
US-01	Registro	Alta	2
US-02	Login	Alta	2
US-03	Crear partida	Alta	2
US-04	Ranking	Media	3

11. Definition of Done
Definición global del proyecto.

12. Plan de Releases y Sprints
Debe incluir la planificación inicial para cada uno de los 5 Sprint solicitados. En cada Sprint se debe indicar el MVP incremental.

Resultado esperado
Al finalizar el Sprint 1, el **ERS/SRS** debería permitir que cualquier desarrollador externo pueda comprender completamente el proyecto, sus requisitos, reglas de negocio, arquitectura, backlog, criterios de aceptación y planificación, sin necesidad de volver a entrevistar al Stakeholder.